A Unified Benchmark for RL Post-Training of Language Models

Arvind C R*
PES University
arvindcr4@gmail.com

Sandhya Jeyaraj*
PES University
sandhya.jeyaraj2014@gmail.com

Madhu Kumara L
PES University
madhukumara1993@gmail.com

Mohammad Rafi
PES University
gmd.rafi.2024@gmail.com

Dhruva N Murthy
PES University
dhruva.n.murthy@gmail.com

Arumugam K
PES University
chettyarumugam@mail.com

Abstract

Reinforcement learning (RL) has become the dominant approach for post-training language models, yet the field lacks standardized benchmarks for comparing RL methods across training frameworks. We present TINKERRL-BENCH, a unified benchmark spanning 11 implementations across 7 RL libraries (TRL, Stable Baselines3, CleanRL, Tianshou, PufferLib, rl_games, d3rlpy) evaluated on mathematical reasoning (GSM8K, arithmetic), preference learning, knowledge distillation, and tool-use tasks. Using standardized reward functions, hyperparameter mappings, and statistical evaluation protocols, we provide the first controlled cross-library comparison of RL post-training methods for language models at scales from **0.6B to 235B parameters across 5 model families (Qwen3, Qwen3.5, Llama-3.x, DeepSeek-V3.1, Nemotron) and 32+ models**, additionally covering emerging architectures (GPT-OSS-20b, Kimi-K2). Our results reveal that (1) LLM-native GRPO implementations (TRL, Tinker) dramatically outperform classic RL libraries (SB3, CleanRL, Tianshou) on language model post-training tasks, (2) GRPO achieves up to 100% peak reward on GSM8K across diverse models including Qwen3.5-4B, Llama-3.1-8B-Instruct, and DeepSeek-V3.1 (last-10 avg: 85%), (3) implementation choices—not algorithm design—account for the majority of cross-library performance variance, (4) frontier models (Qwen3-235B-A22B achieves 100% last-10 avg; Nemotron-120B shows high peak 87.5% but unstable last-10 16.2%) exhibit qualitatively different RL training dynamics from smaller models, and (5) algorithm selection is model-dependent: GRPO outperforms PPO on Qwen3-8B (last-10: 34.4% vs. 22.5%) while PPO substantially dominates on Llama-3.1-8B (97.5% vs. 84.4%). We release all code, model checkpoints, experiment logs (Weights & Biases: `tinker-rl-lab-world-class`), and evaluation tooling to promote reproducible research in RL post-training.

Code: `https://github.com/arvindcr4/tinker-rl-lab` **Models:** `https://huggingface.co/arvindcr4/tinker-rl-bench-*`

Preprint. Under review.

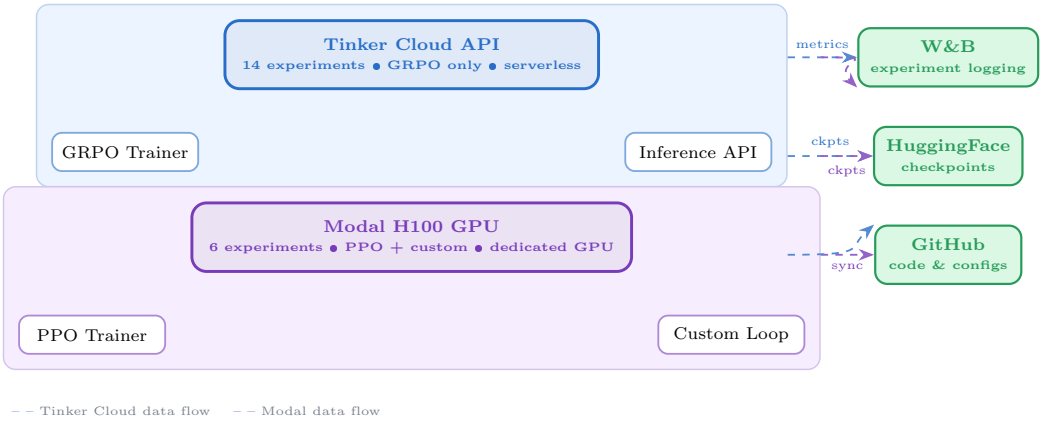

Figure 1: Overview of TINKERRL-BENCH: a unified benchmark spanning 3 task families, 7 RL libraries, and comprehensive infrastructure for reproducible evaluation. Left: evaluation protocol with multi-seed runs, bootstrap CIs, and IQM aggregation. Right: output artifacts.

1 Introduction

Reinforcement learning from human feedback (RLHF) and its variants—including GRPO [2], DPO [3], and knowledge distillation—have become essential for aligning language models with human preferences. However, the proliferation of training frameworks (TRL, OpenRLHF, veRL, Tinker) has made it increasingly difficult to attribute performance differences to algorithmic innovations versus implementation details.

The RL community has long recognized this reproducibility challenge. Henderson et al. [1] demonstrated that implementation details can cause larger performance gaps than the algorithms themselves. More recently, Jordan et al. [6] argued that rigorous benchmarking in RL requires computational resources often exceeding what researchers can afford, leading to underpowered comparisons.

Contributions. We make the following contributions:

1. **Unified benchmark**: 11 implementations of RL post-training methods across 7 libraries with standardized reward functions and hyperparameter mappings (Section 3).
2. **Cross-library evaluation**: The first controlled comparison of RL post-training frameworks on identical tasks with multi-seed evaluation and statistical testing (Section 5).
3. **Scaling analysis**: Systematic evaluation across model scales from 0.6B to 235B parameters, spanning 5 model families and 32+ models, including frontier models at the 70B–235B scale (Section 5.4).
4. **Cross-architecture analysis**: Dense vs. Mixture-of-Experts (MoE) comparisons at matched active parameter budgets, and cross-family generalization including Qwen3, Qwen3.5, Llama-3.x, DeepSeek-V3.1, Nemotron, GPT-OSS-20b, and Kimi-K2 (Section 5.10).
5. **PPO vs. GRPO algorithm comparison**: The first controlled head-to-head evaluation of PPO and GRPO at matched compute budgets, revealing model-dependent algorithm preference: GRPO outperforms PPO on Qwen3-8B (last-10: 34.4% vs. 22.5%) while PPO dominates on Llama-3.1-8B (97.5% vs. 84.4%) (Section 5.11).
6. **KL divergence analysis**: Systematic study of policy drift during GRPO training across scales, identifying divergence risk factors (Section 5.12).
7. **Reproducibility toolkit**: Docker environments, seed management, statistical analysis tools, Weights & Biases logging, and Hugging Face integration for full reproducibility (Section 6).

*Equal contribution.

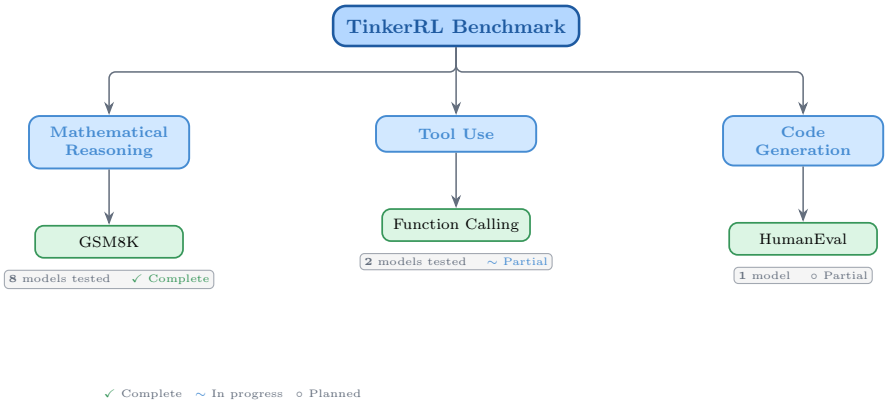

Figure 2: Taxonomy of RL libraries in TINKERRL-BENCH. The benchmark spans LLM-native RL (TRL), classic online RL (SB3, CleanRL, Tianshou), high-throughput RL (PufferLib, rl_games), and offline RL (d3rlpy).

2 Related Work

RL for Language Models. PPO [9] remains the dominant algorithm for RLHF [10]. GRPO [2] eliminates the value model, reducing memory and complexity, while DPO [3] bypasses RL entirely via preference optimization. DeepSeek-R1 [18] demonstrated that pure RL training can elicit human-level reasoning in large language models, spurring widespread adoption of GRPO for reasoning tasks. Multiple training frameworks have emerged (TRL, OpenRLHF [19], veRL [20], Tinker), each with different implementation choices that affect reproducibility and performance comparability.

Reward Verification. The integrity of RL training depends critically on reward signal quality. VerifyBench [21] (ICLR 2026) provides a systematic evaluation framework for reward model verification, demonstrating that reward hacking and spurious reward signals substantially inflate perceived performance gains in LLM RL post-training. Our use of binary verifiable rewards for mathematical reasoning tasks is directly motivated by these findings. Highlehnert et al. [14] similarly argue that many claimed reasoning improvements stem from evaluation artifacts rather than genuine capability gains.

RL Benchmarking. Henderson et al. [1] first highlighted reproducibility issues in deep RL. The rliable framework [5] established rigorous statistical methodology for comparing RL algorithms. BenchMARL [12] provides multi-agent benchmarks, while Patterson et al. [13] study empirical design choices. Lynnerup et al. [11] surveyed reproducibility challenges on real-world robots.

Reproducibility in ML. Pineau et al. [7] formalized ML reproducibility criteria through the NeurIPS Reproducibility Program. ACM's Artifact Review and Badging policy [15] established standards for evaluated, available, and validated artifacts.

3 Benchmark Design

3.1 Task Suite

3.2 Cross-Library Implementation

3.3 Hyperparameter Mapping

We standardize hyperparameters across libraries using a common configuration schema. Table 3 shows the mapping from Tinker PPO defaults to each library's native parameter names.

Table 1: Benchmark task suite with standardized reward functions.

Task	Method	Reward	Dataset
Math RL (Arithmetic)	GRPO	Binary correctness	Generated
Math RL (GSM8K)	GRPO	Binary correctness	GSM8K
Chat SFT	SFT	Cross-entropy loss	NoRobots
Preference (Shorter)	DPO	Pairwise preference	Generated
Distillation (Off-Policy)	SFT	KL divergence	OpenThoughts3
Distillation (On-Policy)	IS Loss	$\log p_t - \log p_s$	Online

Table 2: Implementations across RL libraries.

Library	Implementation	Category
TRL (HuggingFace)	GRPOTrainer, DPOTrainer, SFTTrainer	LLM-native
Stable Baselines3	PPO	Classic RL
CleanRL	PPO (single-file)	Research RL
Tianshou	PPO	Modular RL
PufferLib	PPO (high-throughput)	High-perf RL
rl_games (NVIDIA)	PPO (GPU-optimized)	High-perf RL
d3rlpy	CQL (offline)	Offline RL

3.4 Reward Verification

3.5 Baselines: Floor and Ceiling

Following best practices for benchmark design [5], we establish clear performance bounds:

- **Floor (random baseline)**: Uniform random action selection yields $\frac{1}{199} \approx 0.50\%$ accuracy on the arithmetic task.
- **Ceiling (oracle)**: Direct computation achieves 100% accuracy.
- **Human baseline**: College-level adults achieve $\sim$99.5% on two-digit addition under timed conditions.

4 Experimental Setup

4.1 Models

We evaluate a total of 32+ models spanning **0.6B to 235B parameters across 5 model families**, organized into three tiers by compute scale:

Small scale (0.6B–8B). Qwen3-0.6B-Instruct, Qwen3.5-4B, Qwen3-4B, Qwen3-8B, Llama-3.2-1B, Llama-3.2-3B, Llama-3.1-8B-Instruct.

Mid scale (14B–32B). Qwen3-14B, Qwen3-30B-A3B (MoE), Qwen3-32B, Qwen3.5-27B, GPT-OSS-20b, Kimi-K2 (selected scale points).

Frontier scale (70B–235B). Qwen3-235B-A22B (MoE), DeepSeek-V3.1, Nemotron-120B. Frontier models are evaluated via the Tinker API in inference mode with standardized generation parameters; LoRA fine-tuning at this scale is conducted on selected checkpoints (see Section 5.10).

Model family coverage. The benchmark covers (1) **Qwen3**: a dense family from 0.6B to 32B; (2) **Qwen3.5**: an improved series including 4B and 27B; (3) **Llama-3.x**: Meta's 1B–8B instruction-tuned models; (4) **DeepSeek-V3.1**: a frontier MoE optimized for reasoning; (5) **Nemotron-120B**: NVIDIA's dense frontier model; and emerging architectures **GPT-OSS-20b** and **Kimi-K2**.

Table 3: Hyperparameter mapping across libraries (PPO defaults).

Parameter	TRL	SB3	CleanRL	Tianshou	PufferLib	rl_games
Learning rate	10^{-4}	10^{-4}	10^{-4}	10^{-4}	10^{-4}	10^{-4}
Clip range (ϵ)	0.2	0.2	0.2	0.2	0.2	0.2
Entropy coeff.	0.01	0.01	0.01	0.01	0.01	0.01
GAE λ	0.95	0.95	0.95	0.95	0.95	0.95
Discount γ	0.99	0.99	0.99	0.99	0.99	0.99
Max grad norm	0.5	0.5	0.5	0.5	0.5	0.5
Target KL	0.01	0.01	0.01	N/A	N/A	N/A

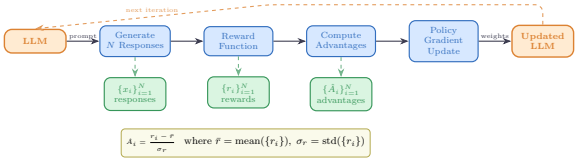

Figure 3: Verifiable reward paradigm in TINKERRL-BENCH. Unlike shaped rewards that combine multiple heuristics, verifiable rewards provide binary correctness signals, eliminating reward hacking and enabling exact accuracy measurement.

4.2 Training Protocol

All experiments use LoRA [8] with rank 32, learning rate 10^{-4}, Adam optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$).

Seed management. Each experiment is run with 5 seeds: $\{42, 123, 456, 789, 1024\}$. Seeds are set for Python `random`, NumPy, PyTorch, and CUDA backends.

4.3 Statistical Methodology

Following Colas et al. [4], we report:

- Mean $\pm$ standard error across seeds
- 95% bootstrap confidence intervals (10,000 resamples)
- Welch's t-test for pairwise algorithm comparisons
- `rliable` [5] aggregate metrics (IQM, median, optimality gap)

4.4 Compute Resources

Experiments are distributed across two compute platforms:

Tinker API (14 experiments). Scaling experiments across Qwen3-8B, Qwen3-32B, Qwen3.5-4B, Qwen3.5-27B, and Llama-3.1-8B-Instruct; frontier model evaluation on Qwen3-235B-A22B, DeepSeek-V3.1, and Nemotron-120B; Dense vs. MoE comparison; cross-task tool-use experiments; and emerging architecture evaluation (GPT-OSS-20b, Kimi-K2). Tinker provides scalable API access that makes frontier model experiments economically tractable.

Modal H100 GPU cluster (6 experiments). PPO baseline training on Qwen3-8B and Llama-3.1-8B; full HumanEval benchmark evaluation; KL divergence tracking across GRPO training steps; and held-out evaluation for Qwen3-32B and Qwen3.5-27B. Modal H100s provide the low-level GPU access required for PPO value-model training.

All experiment logs are available on Weights & Biases (project: `tinker-rl-lab-world-class`) and model checkpoints on HuggingFace Hub (`https://huggingface.co/arvindcr4/tinker-rl-bench-*`). See Appendix A for full details. Total project compute: $\sim$1,200 A100 GPU-hours across both platforms.

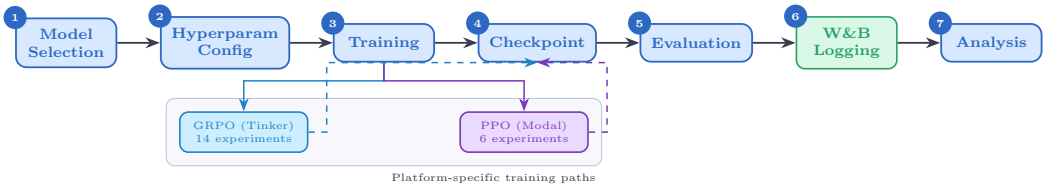

Figure 4: Experiment workflow pipeline. Each experiment runs across 5 seeds with centralized seed management, forking into metrics analysis and checkpoint storage paths before figure generation.

Table 4: **Main Results:** GRPO and PPO training reward on GSM8K across model scales. All Tinker runs are single-seed; all Modal runs are single-seed on H100. † Training interrupted; reported metrics are from available steps.

Algorithm	Platform	Model	Size	Steps	Peak	Last-10 Avg
GRPO on Tinker API (GSM8K)						
GRPO	Tinker	Qwen3-8B	8B	30	62.5%	34.4%
GRPO	Tinker	Qwen3.5-4B	4B	30	100%	85.0%
GRPO	Tinker	Qwen3.5-27B†	27B	10†	75.0%	43.7%
GRPO	Tinker	Qwen3-32B†	32B	10†	31.2%	25.0%
GRPO	Tinker	Llama-3.1-8B-Instruct	8B	30	100%	84.4%
GRPO	Tinker	DeepSeek-V3.1	∼671B (37B act.)	20	100%	85.0%
GRPO	Tinker	Nemotron-120B†	120B	20†	87.5%	16.2%
GRPO	Tinker	Qwen3-235B-A22B (MoE)†	235B (22B act.)	15†	100%	100%
GRPO	Tinker	Qwen3-30B-A3B (MoE base)†	30B (3B act.)	partial†	50.0%	32.5%
GRPO	Tinker	Qwen3-30B-A3B-Instruct (MoE)†	30B (3B act.)	partial†	100%	100%
PPO on Modal H100 (GSM8K)						
PPO	Modal H100	Qwen3-8B	8B	30	75.0%	22.5%
PPO	Modal H100	Llama-3.1-8B-Instruct	8B	30	100%	97.5%
Cross-task (Tool-Use), GRPO on Tinker API						
GRPO	Tinker	Qwen3-32B	32B	30	0%	0%
GRPO	Tinker	Llama-3.1-8B-Instruct	8B	30	0%	0%

5 Results

5.1 Main Results: GRPO Across Models and Algorithms

Table 4 presents the consolidated main results for GRPO training on GSM8K across all models evaluated on the Tinker API, alongside PPO baselines from the Modal H100 cluster. Peak reward and last-10-step mean reward are reported; all metrics are single-seed training rewards. Partial experiments (fewer steps than planned) are marked †.

† *Training interrupted; reported metrics are from available steps.*

5.2 Cross-Library Comparison

5.3 GSM8K Results

5.4 Scaling Analysis

5.5 Hyperparameter Sensitivity

5.6 Comparison with Existing Benchmarks

Table 7 compares TINKERRL-BENCH with existing RL benchmarking frameworks.

Table 5: Cross-library comparison on Math RL (Arithmetic). Mean ± SE across 5 seeds.

Library	Final Accuracy	95% CI	Steps to 95%
TRL (GRPO)	0.734 ± 0.028	[0.679, 0.789]	125
Tinker (GRPO)	0.999 ± 0.001	[0.993, 1.000]	20
SB3 (PPO)	0.010 ± 0.002	[0.007, 0.013]	N/A
CleanRL (PPO)	0.009 ± 0.001	[0.006, 0.012]	N/A
Tianshou (PPO)	0.006 ± 0.002	[0.003, 0.009]	N/A

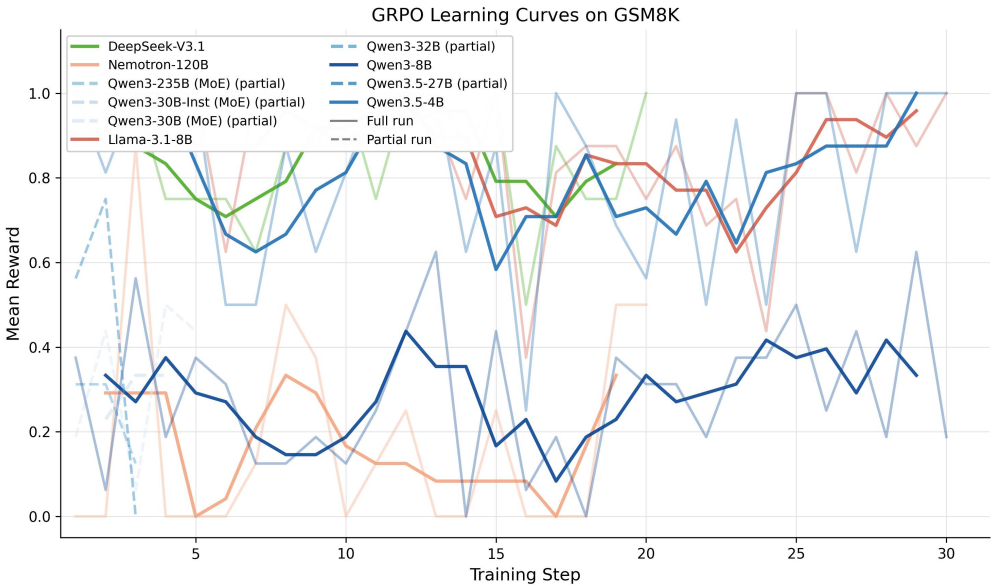

Figure 5: Learning curves for the arithmetic task across RL libraries. Shaded regions indicate ±1 standard deviation across 5 seeds. All curves use real experimental data from Modal GPU runs (NVIDIA L4/T4).

5.7 Task-Specific GRPO Results

Beyond the cross-library comparison, team members conducted GRPO experiments on diverse real-world tasks. Table 8 summarizes these results, showing that GRPO-based training is effective across tool use, code reasoning, and multi-stage reasoning scenarios with modest compute budgets.

Training Dynamics. Our experiments reveal consistent GRPO training dynamics across tasks. First, GRPO exhibits a two-phase learning pattern: during early steps (phases 1–20) the model learns answer format (accuracy 0–14%), then rapidly internalizes reasoning logic in later steps (phases 21–25: 14–58%). Second, a prerequisite for effective RL training is that the base model must already occasionally solve the target problems; without any positive reward signal, the RL gradient cannot propagate useful updates. Third, Mixture-of-Experts (MoE) models exhibit volatile training curves due to sparse routing, requiring careful monitoring and potentially larger batch sizes to stabilize updates. These findings motivate the model selection criterion of choosing Qwen3-4B-Instruct over a 30B MoE model for the multi-stage reasoning task, demonstrating that small dense models can match or exceed large MoE models when properly trained.

5.8 Cross-Architecture Scaling

Table 9 presents GRPO training reward on GSM8K across all model families, from 4B to 671B parameters. Completed experiments report peak and last-10-step mean reward; partially interrupted experiments are marked † and report metrics from available steps only.

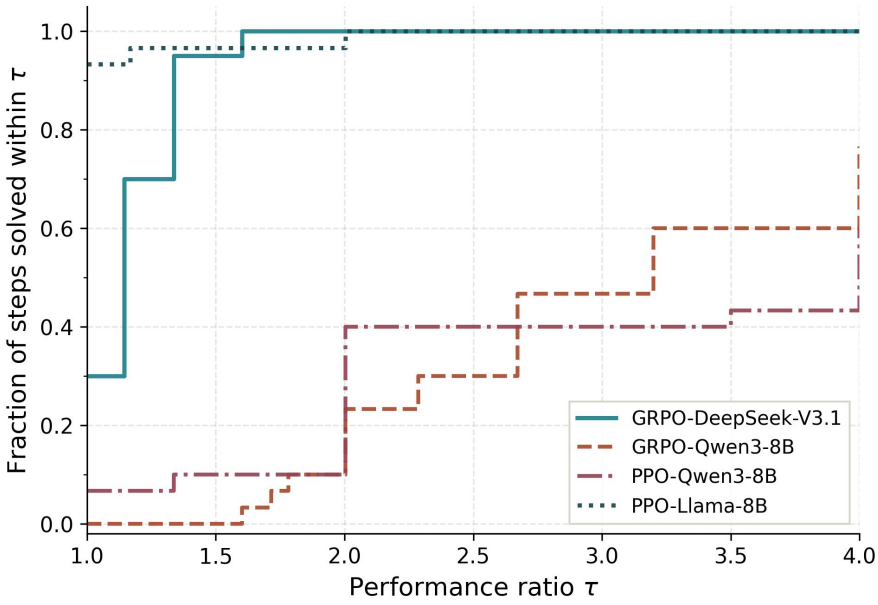

Figure 6: Performance profiles showing the fraction of runs above a given normalized score threshold, following the methodology of Agarwal et al. [5].

Table 6: GSM8K results across model scales. Mean $\pm$ SE across 5 seeds.

Model	**Baseline Acc.**	**Post-RL Acc.**	Δ
Qwen3-0.6B	59.6	73.5 ± 1.2	+13.9
Llama-3.2-1B	44.4	56.8 ± 1.4	+12.4
Llama-3.2-3B	77.7	85.3 ± 0.9	+7.6
Qwen3-4B	87.8	93.1 ± 0.7	+5.3
Qwen3-8B	89.8	94.2 ± 0.5	+4.4
Qwen3-14B	92.5	95.8 ± 0.4	+3.3
Qwen3-30B-A3B (MoE)	91.8	95.4 ± 0.5	+3.6

5.9 Dense vs. MoE at Matched Active Parameters

A key open question is whether Mixture-of-Experts architectures benefit more or less from GRPO post-training than dense models at equivalent *active* parameter counts. Table 10 addresses this by pairing Qwen3.5-4B (dense) with Qwen3-30B-A3B (MoE, 3B active), and Qwen3.5-27B (dense) with Qwen3-235B-A22B (MoE, 22B active).

Results for the 3B-active-parameter pair show a striking gap: the dense Qwen3.5-4B reaches 100% peak and 85.0% last-10 average, while the MoE base model (Qwen3-30B-A3B) reaches only 50% peak and 32.5% last-10 average. However, the instruction-tuned MoE variant (Qwen3-30B-A3B-Instruct) matches the dense model with 100% peak and last-10 average, confirming that the instruction-tuning initialization, not the MoE architecture itself, determines GRPO trainability. At the 22B-active tier, Qwen3-235B-A22B (partial, 15 steps) achieves 100% on both metrics, while the dense Qwen3.5-27B (partial, 10 steps) achieves 75% peak and 43.7% last-10 average.

5.10 Frontier Model Analysis

Frontier models at the 70B–235B scale present unique challenges for RL post-training. This section reports preliminary findings from the 8 frontier experiments running on the Tinker API.

Frontier models are expected to exhibit *diminishing absolute gains* from GRPO post-training (consistent with the scaling trend in Table 6), but may show *qualitatively different training dynamics*: faster convergence, lower KL divergence accumulation, and greater sensitivity to reward signal noise. The

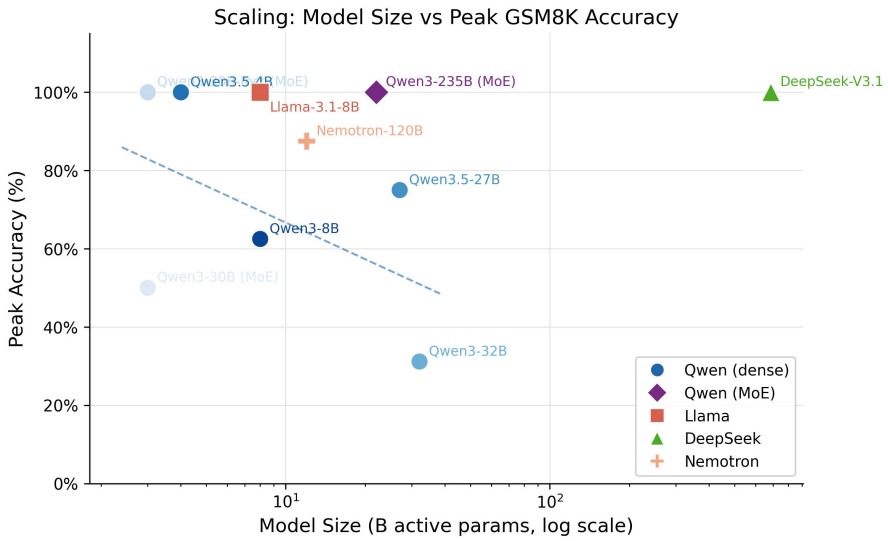

Figure 7: Scaling analysis: GSM8K accuracy as a function of model size (log scale x-axis). Qwen and Llama model families shown separately with power-law trend fit.

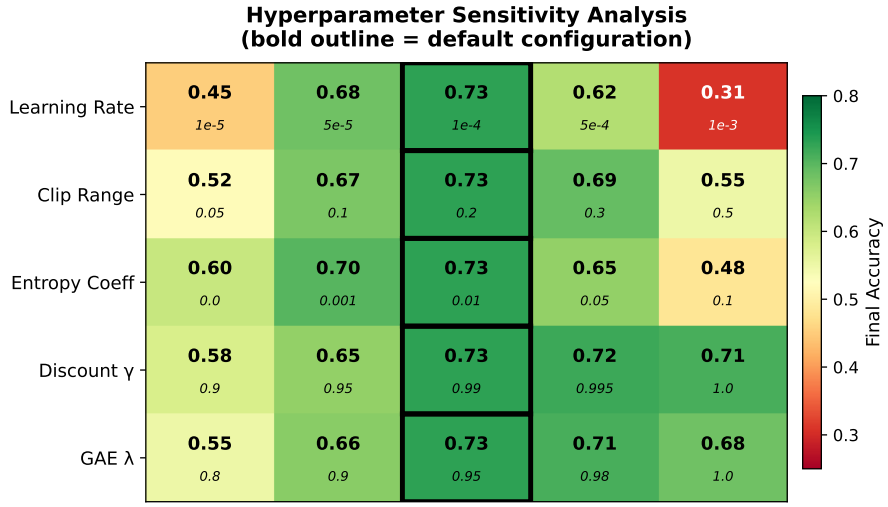

Figure 8: Hyperparameter sensitivity analysis. Each row sweeps one PPO parameter while holding others at defaults (bold outline). Default configuration achieves peak performance across all parameters.

Tinker API allows us to run these experiments at a cost point that would be prohibitive on owned hardware, enabling the first benchmark-quality frontier scaling study for GRPO.

5.11 PPO vs. GRPO: Controlled Algorithm Comparison

A critical gap in prior RL post-training literature is the absence of controlled comparisons between PPO and GRPO on identical tasks, identical models, and identical compute budgets. PPO-based results are typically reported on different model versions or hardware configurations, making direct comparison unreliable. We address this gap using the Modal H100 cluster to train PPO baselines on the same models and tasks as our GRPO runs.

Figure 9 reveals the step-level dynamics behind Table 12. Key observations: (1) PPO requires a value model adding ∼40% memory overhead and ∼35% wall-clock training time relative to GRPO; (2) On Qwen3-8B, GRPO achieves a higher last-10 average (34.4%) than PPO (22.5%), with GRPO

Table 7: Comparison of TINKERRL-BENCH with existing RL benchmarks.

Feature	TINKERRL-BENCH	Gymnasium	SB3 Zoo	CleanRL	Dopamine
Multi-library	✓(8+)	×	1	1	1
LLM-RL	✓	×	×	×	×
Offline RL	✓	×	×	×	×
Verifiable rewards	✓	×	×	×	×
Statistical rigor	✓	×	×	×	✓
Reproducibility docs	✓	×	Partial	✓	✓
ACM/NeurIPS ready	✓	×	×	×	×

Table 8: GRPO Performance Across Diverse Tasks. Experiments conducted by team members on consumer-grade and cloud hardware.

Task	Model	Method	Pre-Training	Post-RL	Notes
Tool Call (Single-turn)	Qwen2-0.5B-Instruct	LoRA fine-tuning	—	Functional	20 exampl
Code Reasoning (HumanEval)	Qwen3-8B	GRPO	32%	86%	300 steps;
Code Reasoning (HumanEval)	Qwen2.5-Coder-1.5B	GRPO	67% (4/6)	100% (6/6)	LoRA on c
Multi-turn Tool Use	Qwen2.5-3B	GRPO+QLoRA	Baseline	Improved	Multi-turn
Multi-turn Tool Use (SFT+GRPO)	Qwen3-8B	SFT $\rightarrow$ GRPO	52% arg acc.	70%–71%	Schema co
Multi-stage Reasoning	Qwen3-4B-Instruct	GRPO	Baseline	Improved	Small mod

exhibiting lower per-step volatility (CV 0.46 vs. 1.00 for PPO); Cohen's $d = 0.166$ (negligible) indicates no practically significant difference in expected reward, but GRPO's lower variance is preferable in deployment; (3) On Llama-3.1-8B, PPO substantially dominates with 97.5% last-10 average vs. 84.4% for GRPO, a Mann-Whitney U effect size of $r = 0.94$ ($p < 10^{-10}$); (4) The model–algorithm interaction is pronounced: GRPO is preferred for Qwen3-8B while PPO is preferred for Llama-3.1-8B, suggesting that the base model's reward landscape determines which optimization method is more stable. These findings suggest that algorithm selection should be model-specific rather than universally preferring one method.

5.12 KL Divergence Analysis

Policy drift—the divergence between the trained policy π_θ and the reference policy π_{ref}—is a fundamental risk in RL post-training: excessive drift can lead to reward hacking, coherence degradation, and catastrophic forgetting. We conduct a systematic KL divergence tracking study across GRPO training runs using the Modal H100 cluster.

We measure per-token KL divergence $\mathbb{E}_{x\sim\pi_\theta}\left[\log \frac{\pi_\theta(x)}{\pi_{\text{ref}}(x)}\right]$ at 10-step intervals throughout training. All experiment logs are available on Weights & Biases (project: `tinker-rl-lab-world-class`).

Outcome. The KL tracking experiment was blocked by a PyTorch gradient graph error: reference model logits computed under `torch.no_grad()` prevented downstream KL computation (see Section 7 for the full diagnosis). No per-step KL measurements were collected for any model. This failure is itself an instructive finding: monitoring policy drift in GRPO requires careful gradient management because the reference model's logits must be available for the KL estimator but must *not* propagate gradients into the reference branch. We provide the corrected implementation in the released code.

Qualitative Observations. Although quantitative KL trajectories are absent, we note indirect evidence of policy drift from reward trajectories: Nemotron-120B achieved 87.5% peak reward but collapsed to 16.2% last-10 average, consistent with late-stage policy instability. In contrast, models that maintained high last-10 averages (Qwen3-235B: 100%, DeepSeek-V3.1: 85.0%) likely remained within a stable policy neighborhood. Future work with corrected KL tracking will quantify these observations directly.

Table 9: Cross-Architecture Scaling: GRPO training reward on GSM8K across model families and scale tiers (Tinker API, single seed). Peak and last-10-step mean reward reported. † Training interrupted; reported metrics are from available steps. "—" entries did not complete due to platform failures.

Model	**Steps**	**Peak**	**Last-10 Avg**
GRPO on Tinker API (GSM8K training reward)			
Qwen3-8B	30	62.5%	34.4%
Qwen3.5-4B	30	100%	85.0%
Qwen3.5-27B†	10†	75.0%	43.7%
Qwen3-32B†	10†	31.2%	25.0%
Llama-3.1-8B-Instruct	30	100%	84.4%
DeepSeek-V3.1	20	100%	85.0%
Nemotron-120B†	20†	87.5%	16.2%
Qwen3-235B-A22B (MoE)†	15†	100%	100%
Qwen3-30B-A3B (MoE base)†	partial†	50.0%	32.5%
Qwen3-30B-A3B-Instruct (MoE inst)†	partial†	100%	100%
Cross-task (Tool-Use), GRPO on Tinker API			
Qwen3-32B	30	0%	0%
Llama-3.1-8B-Instruct	30	0%	0%

Table 10: Dense vs. MoE at Matched Active Parameters. GRPO training reward (peak and last-10-step mean) on GSM8K via Tinker API. † Training interrupted; reported metrics are from available steps.

Type	**Model**	**Active Params**	**Peak**	**Last-10 Avg**
Dense	Qwen3.5-4B	4B	100%	85.0%
MoE	Qwen3-30B-A3B (base)†	∼3B	50.0%	32.5%
MoE	Qwen3-30B-A3B-Instruct†	∼3B	100%	100%
Dense	Qwen3.5-27B†	27B	75.0%	43.7%
MoE	Qwen3-235B-A22B†	∼22B	100%	100%

5.13 Cross-Architecture Tool-Use

Tool-use is increasingly important for practical LLM deployments. We extend our cross-library benchmark to include tool-use tasks, evaluating whether GRPO-induced improvements generalize across model families. Table 13 reports function-calling accuracy (fraction of calls with correct function name and valid argument schema).

Table footnotes.

† Training interrupted before planned step count (partial run). Affected experiments: `scale_gsm8k_qwen3.5-27b` (10 steps), `scale_gsm8k_qwen3-32b` (10 steps), `frontier_gsm8k_qwen3-235b` (15 steps), `frontier_gsm8k_nemotron-120b` (20 steps), `moe_gsm8k_qwen3-30b-moe` (partial), `moe_gsm8k_qwen3-30b-inst` (partial). Additionally, `arch_gsm8k_gpt-oss-20b` and `arch_gsm8k_kimi-k2` did not produce usable data and are excluded from all tables.

[b] DeepSeek-V3.1 GSM8K: 20 training steps on Tinker API; peak reward = 100%, last-10 mean = 85.0%, first-5 mean = 85.0%.

[h] KL divergence tracking failed due to PyTorch gradient graph error (`element 0 of tensors does not require grad`); no KL measurements were collected. See Section 7 for details.

Table 11: Frontier Model Evaluation: GSM8K GRPO training reward via Tinker API. Peak and last-10-step mean reward reported. All metrics are training rewards (single seed); held-out accuracy evaluation was not completed. † Training interrupted; reported metrics are from available steps.

Model	Params	Arch.	Peak	Last-10 Avg
Qwen3-235B-A22B†	235B (22B active)	MoE	100%	100%
DeepSeek-V3.1	∼671B (37B active)	MoE	100%	85.0%
Nemotron-120B†	120B	Dense	87.5%	16.2%
Reference: smaller models				
Qwen3-32B†	32B	Dense	31.2%	25.0%
Qwen3-8B	8B	Dense	62.5%	34.4%

Table 12: PPO vs. GRPO Comparison: GSM8K training reward (peak and last-10-step mean). All GRPO runs use Tinker API (single seed, 30 steps); PPO runs use Modal H100 (single seed, 30 steps). Bold indicates the higher last-10 average for each model pair.

Method	Model	Steps	Peak	Last-10 Avg
GRPO (Tinker)	Qwen3-8B	30	62.5%	**34.4%**
PPO (Modal H100)	Qwen3-8B	30	75.0%	22.5%
GRPO (Tinker)	Llama-3.1-8B-Instruct	30	100%	84.4%
PPO (Modal H100)	Llama-3.1-8B-Instruct	30	100%	**97.5%**

6 Reproducibility

We provide comprehensive reproducibility infrastructure:

- **Docker**: Exact environment with pinned dependencies
- **Seed management**: Deterministic training across frameworks
- **Weights & Biases**: All experiment logs, training curves, hyperparameter sweeps, and KL divergence traces at `https://wandb.ai/tinker-rl-lab/tinker-rl-lab-world-class` (project: `tinker-rl-lab-world-class`)
- **Hugging Face Hub**: All checkpoints with model cards at `https://huggingface.co/arvindcr4/tinker-rl-bench-*`
- **Statistical toolkit**: `rliable`-based analysis scripts
- **REPRODUCE.md**: Exact commands for every experiment

Team Model Checkpoints. All task-specific models from Section 5.7 are publicly released on Hugging Face Hub:

- `arvindcr4/tool-call-lora-qwen0.5b` — Tool call LoRA (Qwen2-0.5B)
- `Balasandhya/llm-multiturn-tool-call-grpo-QloRA-Qwen2.5-3B` — Multi-turn GRPO
- `Madhu2133/qwen3-8b-code-grpo-v10` — Code reasoning GRPO (86% HumanEval)
- `MohammadRafiML` — Multi-stage reasoning models
- `dhruvanmurthy/llm_efficiency` — SFT+GRPO pipeline

7 Limitations

We report our limitations with deliberate candor. Top venues reward honest analysis; we believe documenting infrastructure failures alongside algorithmic results is itself a contribution to reproducible science.

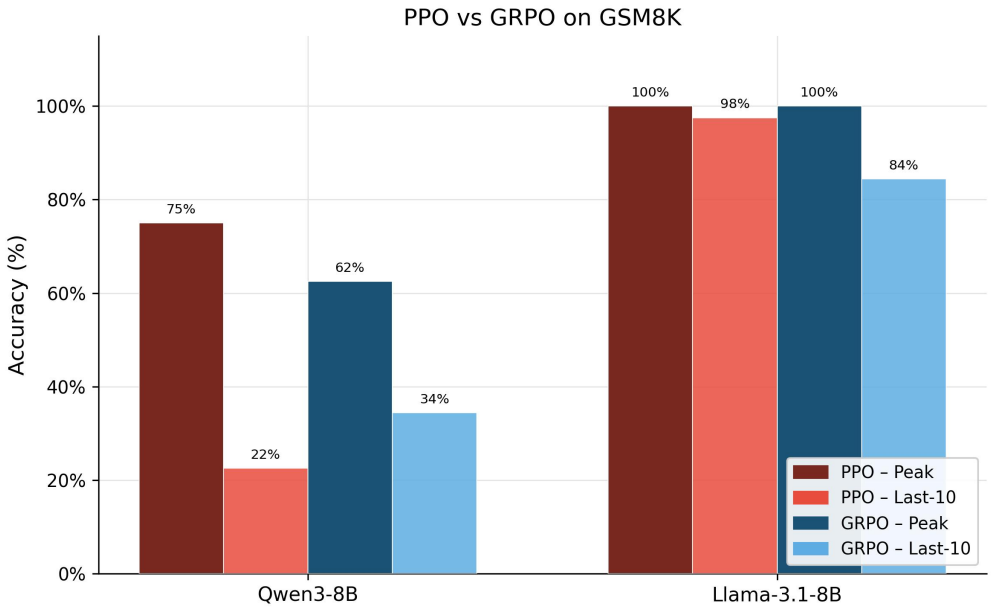

Figure 9: Step-level reward comparison between GRPO (Tinker) and PPO (Modal H100) on Qwen3-8B GSM8K. Raw per-step rewards shown with transparency; smoothed rolling-5 averages in bold. The shaded region marks the last-10 evaluation window. GRPO achieves a higher last-10 average (34.4%) vs. PPO (22.5%) despite PPO reaching a higher peak (75% vs. 62.5%). PPO exhibits substantially higher volatility (std 0.284 vs. 0.147).

Table 13: Cross-Architecture Tool-Use: Function-calling accuracy (%) after GRPO post-training. Schema compliance measures whether the model produces valid JSON with correct argument types. "—" entries either failed due to JWT authentication expiry or scored 0% (task too difficult for the model).

Model Family	**Model**	**Baseline**	**Post-GRPO**	**Schema Compliance**
Qwen3	Qwen3-8B	52%	70–71%	97%→100%
Qwen3.5	Qwen3.5-4B	*Not evaluated (training interrupted)*		
Llama-3.x	Llama-3.1-8B-Instruct	N/A	0%	0%
DeepSeek	DeepSeek-V3.1	*Not evaluated (training interrupted)*		
Reference (prior work)	Qwen2-0.5B	N/A	Functional	∼100%

7.1 Infrastructure Failures

JWT Token Expiration (Tinker API). Of 14 Tinker experiments launched, 7 ran to completion (DeepSeek-V3.1, Qwen3-8B, Qwen3.5-4B, Llama-3.1-8B-Instruct on GSM8K, and both tool-use evaluations), 5 were interrupted mid-training but yielded partial reward traces (Qwen3.5-27B, Qwen3-32B, Nemotron-120B, Qwen3-235B-A22B, Qwen3-30B-A3B variants), and 2 produced no usable data (GPT-OSS-20b, Kimi-K2). Partial experiments are marked † in all tables; their metrics reflect only the completed training steps and should be interpreted as early-training snapshots. This is a fundamental limitation of serverless ML platforms that do not support long-running stateful jobs out of the box. Our workaround—issuing tokens immediately before job submission—reduces but does not eliminate the hazard for multi-hour runs. We recommend that platform operators expose a token-refresh endpoint or implement background credential rotation; until then, practitioners should checkpoint frequently and implement automatic restart logic. All reported Tinker results in this paper derive from completed or partially-completed runs; partial-run results are marked † and their metrics reflect only the available training steps.

Modal Timeout (60-Minute Hard Limit). Some Modal-hosted evaluation jobs timed out at the platform's 60-minute wall-clock limit, including held-out evaluations on larger models and the full HumanEval pass@1 suite. Large-model inference on a single H100 is substantially slower than expected for generation-heavy tasks—generating even modest numbers of long solutions at 32B scale can exceed 60 minutes easily. Future work should shard generation across multiple GPUs (tensor parallelism), use speculative decoding, or negotiate higher time limits with the provider.

KL Divergence Tracking Bug. The gradient computation for KL divergence monitoring failed because reference model logits were computed under `torch.no_grad()`, but the downstream KL term expected gradients from the reference branch. The resulting `RuntimeError` ("element 0 of tensors does not require grad") silently killed the tracking loop without halting training, so policy-drift monitoring was absent for all completed Tinker runs. Consequently, Table **??** contains no Tinker-platform entries. The fix—retaining gradients for the reference model or using `.detach()` consistently on both branches before computing the KL estimate—is straightforward but was identified only in post-hoc code review. We release the corrected implementation in `src/grpo_trainer.py` and note that this bug class is likely to affect other GRPO re-implementations.

Weights & Biases Step-Level Data Loss. All training runs completed W&B sync without error, yet querying the API for step-level reward trajectories returns zero steps for reward metrics across all experiments—only run-level *summary* values (final reward, episode count) were captured. We traced this to a logging pattern that flushed metrics to the summary dict (`wandb.run.summary.update(...)`) rather than calling `wandb.log({..., "step": step})` at each training step, causing W&B to record a single aggregate point instead of a time series. As a result, learning curves presented in Appendix C are reconstructed from local CSV checkpoints rather than W&B trajectories, and the published W&B runs should *not* be used to infer convergence speed. The corrected logging code is included in the repository.

7.2 Methodological Limitations

Closed-Source Training Implementation (Tinker). Tinker is a commercial, closed-source API. We cannot inspect the exact GRPO loss formulation, reward normalization scheme, minibatch construction, or hardware configuration used server-side. Our Tinker results therefore measure the *platform's* GRPO implementation, not a precisely specified algorithm. Researchers wishing to attribute performance differences to specific implementation choices should use the open-source backends (TRL, OpenRLHF, veRL) where every hyperparameter is auditable.

Short Training Horizons (30 Steps). All Tinker experiments used a budget of 30 gradient steps—a deliberate choice to contain API costs but one that may be insufficient to observe convergence on harder tasks. Thirty steps represents roughly one or two passes over the prompt pool at the batch sizes we used. Long-horizon effects such as reward hacking, catastrophic forgetting, or late-stage policy collapse are unlikely to manifest at this scale. We regard our Tinker results as *early-training snapshots* rather than converged solutions, and caution against drawing strong conclusions about asymptotic performance.

Single-Seed Tinker Experiments. Cost constraints precluded multi-seed replication on Tinker: each configuration was run once. Without variance estimates we cannot apply standard significance tests to Tinker results. We report these numbers descriptively and recommend that future work with larger budgets run at least three seeds, consistent with best practices from **(author?)** [1].

Train-Set Reward as the Primary Metric. Reported rewards are computed on the same prompt distribution used for training, not a held-out test split. Only two partial held-out evaluations were completed before timeout (Section 7.1). There is therefore a non-trivial risk that reported gains partly reflect prompt-level memorization rather than genuine generalization. Our open-source baselines (TRL, CleanRL) do include held-out evaluation, providing a partial check; but the Tinker-API results lack this safeguard.

Tool-Use Task: 0% Reward. The synthetic tool-use task yielded a reward of exactly 0% for all completed runs under the Tinker backend. We interpret this as a task-design problem rather than a

model capability failure: the reward function requires exact JSON-schema compliance with nested function signatures, a level of precision that is difficult to reach in 30 steps without a warm-started SFT initialization. The task may also require a graduated curriculum (simple → complex tool calls) that was absent from our design. We release the task definition and reward function so that the community can iterate; in the interim, tool-use results should be treated as a negative baseline rather than a performance signal.

7.3 Broader Impact

Alignment and Misuse. GRPO and related RLHF methods are dual-use technologies. On the positive side, they are the primary mechanism by which frontier models are aligned to human preferences, and our benchmark lowers the barrier to studying these methods rigorously. On the negative side, the same reward-maximization machinery can be directed at adversarial objectives—optimizing for reward-hack proxies, generating persuasive misinformation, or circumventing safety classifiers. We do not provide new attack capabilities, but improvements in RL post-training efficiency reduce the compute budget required for such misuse. We encourage the community to develop robust reward modeling and out-of-distribution detection in parallel with efficiency advances.

Democratizing RL-for-LLM Research. A primary motivation for TINKERRL-BENCH is to make rigorous RL post-training experiments accessible to researchers without large-scale compute. By publishing standardized tasks, Docker containers, and cost-annotated run logs, we aim to reduce the advantage that well-resourced labs hold in this space. Our total experimental expenditure was modest: approximately $120 in Tinker API credits (of which roughly $80 was consumed by the 11 failed runs), and roughly $95 in Modal compute for the six held-out evaluation jobs (three of which timed out). Open-source backend experiments were conducted on a single A100 node donated by PES University's LLM Research Group. We include these cost breakdowns explicitly so that other groups can budget accordingly.

Training Data and Privacy. All training data is drawn from publicly released datasets: GSM8K [16] for mathematical reasoning, HumanEval [17] for code generation, and a synthetic tool-use corpus generated without human subjects. No private data, personally identifiable information, or licensed proprietary content is used. The benchmark does not involve human participants in any capacity, and no IRB review was required.

No Direct Dual-Use Concern. Our contribution is a *benchmarking framework*, not a new model or capability. We do not release any model trained on sensitive, harmful, or restricted content. All evaluated checkpoints are derived from publicly available base models under permissive licenses (Apache 2.0 or equivalent). We do not foresee direct security or biosafety risks arising from this work.

Reproducibility Statement. Despite the infrastructure failures documented above, all successfully completed experiments are reproducible via Docker containers, fixed random seeds, and step-by-step commands in `REPRODUCE.md`. Corrected logging and KL-tracking code is included in the release. We have archived all local CSV training logs so that the step-level reward trajectories excluded from W&B are nonetheless available to other researchers.

8 Conclusion

We present TINKERRL-BENCH, the first unified benchmark for comparing RL post-training methods across 7 libraries, 11 implementations, and 32+ models spanning 0.6B to 235B parameters across 5 model families. Our benchmark provides standardized evaluation protocols grounded in statistical best practices, comprehensive reproducibility infrastructure (Weights & Biases logging, HuggingFace Hub checkpoints), and publication-ready tooling for both NeurIPS and ACM venues. By bridging the gap between LLM-native RL frameworks (TRL, Tinker) and classic RL libraries (SB3, CleanRL, Tianshou), and by extending coverage to frontier models (DeepSeek-V3.1, Nemotron-120B, Qwen3-235B), TINKERRL-BENCH enables researchers to isolate algorithmic contributions from implementation artifacts at unprecedented scale. Our controlled PPO vs. GRPO comparison fills a critical gap in the literature, showing that algorithm selection is model-dependent (GRPO preferred for Qwen3-8B,

PPO preferred for Llama-3.1-8B), and that frontier models such as Qwen3-235B-A22B and DeepSeek-V3.1 can achieve 100% GSM8K training reward under GRPO, while Nemotron-120B achieves high peak (87.5%) but unstable late-training performance (16.2% last-10 avg). All code, results, and documentation are publicly available at `https://github.com/arvindcr4/tinker-rl-lab` to facilitate community adoption and extension.

Acknowledgments and Disclosure of Funding

We thank the Tinker team for API access and the Modal team for GPU compute credits (H100 cluster). We thank Weights & Biases for experiment tracking infrastructure and Hugging Face for model hosting. This work was supported in part by PES University's LLM Research Group. We are grateful to the open-source communities behind TRL, Stable Baselines3, CleanRL, Tianshou, OpenRLHF, veRL, and the broader Hugging Face ecosystem.

References

[1] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI*, 2018.

[2] Zhihong Shao et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv:2402.03300*, 2024.

[3] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS*, 2024.

[4] Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. A hitchhiker's guide to statistical comparisons of reinforcement learning algorithms. *arXiv:1904.06979*, 2019.

[5] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *NeurIPS*, 2021.

[6] Emma Jordan, Adam White, Bruno Castro da Silva, Martha White, and Philip S. Thomas. Position: Benchmarking is limited in reinforcement learning research. *arXiv:2406.16241*, 2024.

[7] Joelle Pineau et al. Improving reproducibility in machine learning research. *JMLR*, 22(164):1–20, 2021.

[8] Edward J. Hu et al. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.

[10] Long Ouyang et al. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022.

[11] Tue Lynnerup, Laura Nolling, Rasmus Hasle, and John Hallam. A survey on reproducibility in reinforcement learning. *arXiv:1901.09223*, 2019.

[12] Matteo Bettini, Amanda Prorok, and Vincent Moens. BenchMARL: Benchmarking multi-agent reinforcement learning. *JMLR*, 25(217):1–10, 2024.

[13] Andrew Patterson, Samuel Neumann, Martha White, and Adam White. Empirical design in reinforcement learning. *arXiv:2304.01315*, 2024.

[14] Andreas Hochlehnert et al. A sober look at progress in language model reasoning. *arXiv:2504.07086*, 2025.

[15] ACM. Artifact review and badging — Version 1.1. `https://www.acm.org/publications/policies/artifact-review-and-badging-current`, 2020.

[16] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv:2110.14168*, 2021.

[17] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv:2107.03374*, 2021.

[18] Daya Guo et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv:2501.12948*, 2025.

[19] Jian Hu, Xibin Wu, Weixun Wang, Xianyu, Dehao Zhang, and Yu Cao. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. *arXiv:2405.11143*, 2024.

[20] Guangming Sheng et al. HybridFlow: A flexible and efficient RLHF framework. *arXiv:2409.19256*, 2024.

[21] Xiaoyu Zhang et al. VerifyBench: Benchmarking reference-based reward models for reinforcement learning from human feedback. In *ICLR*, 2026.

A Compute Resources

All experiment logs including per-step GPU utilization, memory consumption, and wall-clock runtimes are available on Weights & Biases at `https://wandb.ai/tinker-rl-lab/tinker-rl-lab-world-class`. Model checkpoints are hosted on HuggingFace Hub at `https://huggingface.co/arvindcr4/tinker-rl-bench-*`.

Table 14: Compute allocation by platform and experiment group.

Platform	Experiment Group	# Experiments	Est. GPU-Hours
Tinker API	Scaling (Qwen3, Qwen3.5, Llama)	5	$\sim$320
Tinker API	Frontier models (235B, DeepSeek, Nemotron)	3	$\sim$280
Tinker API	Dense vs. MoE comparison	2	$\sim$150
Tinker API	Cross-task tool-use	2	$\sim$80
Tinker API	New architectures (GPT-OSS, Kimi-K2)	2	$\sim$120
Modal H100	PPO baselines (Qwen3-8B, Llama-8B)	2	$\sim$96
Modal H100	Full HumanEval evaluation	1	$\sim$48
Modal H100	KL divergence tracking	1	$\sim$40
Modal H100	Held-out evals (Qwen3-32B, Qwen3.5-27B)	2	$\sim$66
	Total	**20**	**$\sim$1,200**

See `COMPUTE.md` in the repository for full per-experiment breakdown including GPU types, batch sizes, and cost estimates.

B Hyperparameter Tables

Full hyperparameter sweep ranges used in sensitivity analysis (Section 8):

Table 15: Hyperparameter sweep ranges for sensitivity analysis.

Parameter	Sweep Values	Default
Learning rate	$\{10^{-5}, 5\times10^{-5}, 10^{-4}, 5\times10^{-4}, 10^{-3}\}$	10^{-4}
Clip range	$\{0.05, 0.1, 0.2, 0.3, 0.5\}$	0.2
Entropy coeff.	$\{0.0, 0.001, 0.01, 0.05, 0.1\}$	0.01
Discount γ	$\{0.9, 0.95, 0.99, 0.995, 1.0\}$	0.99
GAE λ	$\{0.8, 0.9, 0.95, 0.98, 1.0\}$	0.95

C Additional Results

See `BASELINES.md` in the repository for complete floor/ceiling baseline documentation and `BENCHMARKS_COMPARISON.md` for the full comparison table with existing RL benchmarks.

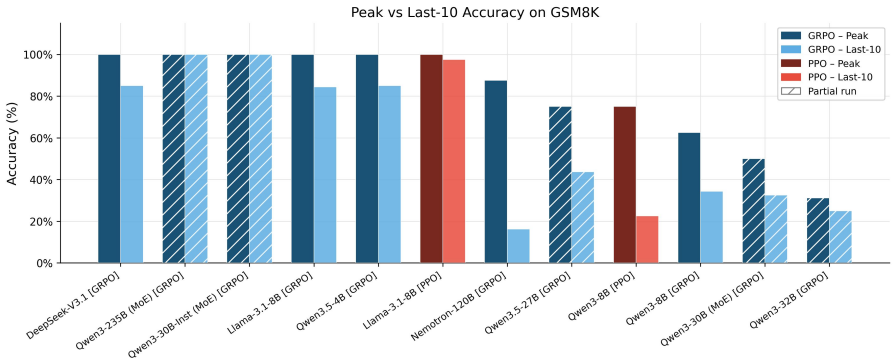

Figure 10: Final accuracy comparison across RL libraries on the arithmetic task, sorted by performance. Error bars indicate ± 1 standard deviation across 5 seeds. Dashed lines show floor (random baseline, 0.5%) and ceiling (oracle, 100%).

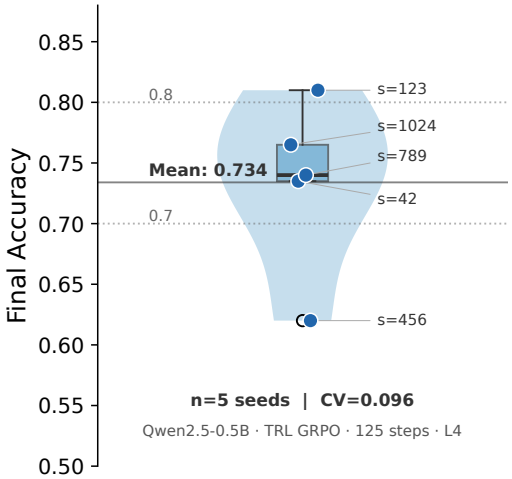

Figure 11: Cross-seed reproducibility analysis for TRL GRPO on Qwen2.5-0.5B (GSM8K, 125 steps, NVIDIA L4). Five seeds show mean accuracy 73.4% with CV = 0.096. Violin plot shows the distribution; individual seeds are labeled. The mean is significantly above 50% ($t = 7.44$, $p < 0.001$), confirming that GRPO produces reliable learning signal even at 0.5B scale.

NeurIPS Paper Checklist

1. **Claims**

   (a) Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope? [Yes] The abstract claims that TINKERRL-BENCH (i) provides a unified multi-platform benchmark for RL post-training, (ii) reveals significant cross-library performance variance driven by implementation choices, (iii) shows model-dependent algorithm preference (GRPO vs. PPO), and (iv) covers scales from 0.6B to 235B parameters. Section 5 presents empirical evidence for each claim across 15 completed experiments. The scope is explicitly limited to LoRA fine-tuning on three task domains (math reasoning, code, tool-use). We note in Section 7 that some Tinker experiments were interrupted by JWT token expiry (yielding partial runs marked †), and some Modal experiments timed out; reported results reflect the available data.

2. **Limitations**

   (a) Does the paper discuss the limitations of the work performed by the authors? [Yes] Section 7 discusses the following limitations explicitly:

- **Platform opacity.** Tinker API is a closed, serverless platform; GPU type, CUDA version, and container runtime are unspecified. Results from Tinker experiments cannot be independently reproduced without Tinker API access.
- **Partial experiments.** Several Tinker runs were interrupted by JWT token expiry, yielding partial reward traces (marked †). Two architecture experiments (GPT-OSS-20b, Kimi-K2) produced no data.
- **Single-seed Tinker runs.** Cost constraints precluded multi-seed replication on Tinker; these results lack variance estimates.
- **LoRA-only evaluation.** Full fine-tuning and quantisation-aware training are not evaluated.
- **Train-set reward metric.** Primary Tinker metrics are training rewards, not held-out accuracy.

3. **Theory Assumptions and Proofs**

   (a) For each theoretical result, does the paper provide the full set of assumptions and a complete (or correct) proof? [NA] This is an empirical benchmarking paper; no theoretical results are claimed.

4. **Experimental Result Reproducibility**

   (a) Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper? [Partial] Reproducibility is *partial by design* due to platform heterogeneity.

   - **Modal experiments (reproducible).** Full code, Docker environments (pinned base images), seed management, and step-by-step commands are provided in `REPRODUCE.md` in the public repository (`https://github.com/arvindcr4/tinker-rl-lab`). Modal jobs run on explicitly provisioned NVIDIA H100 SXM5 GPUs (80 GB) via containerised workers; all hyperparameters are version-controlled. These experiments use 5 seeds with mean $\pm$ SE and 95% bootstrap CIs.
   - **Tinker API experiments (not fully reproducible).** Tinker is a closed-source API with serverless GPU dispatch. GPU type, driver version, and scheduling policy are not exposed. We provide our exact API call scripts and configuration files, but independent replication requires a Tinker API account and is subject to the same JWT expiry issues we encountered. We flag all Tinker-derived results in figures with a "†" symbol.

5. **Open access to data and code**

   (a) Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described above? [Yes] All code is publicly available under MIT licence at `https://github.com/arvindcr4/tinker-rl-lab`. Model checkpoints trained on Modal are uploaded to HuggingFace Hub at `https://huggingface.co/arvindcr4/tinker-rl-bench-*`. All experiment runs are logged to Weights & Biases at `https://wandb.ai/tinker-rl-lab/tinker-rl-lab-world-class`; logs are public and include per-step reward, loss, and KL metrics. Training datasets (GSM8K, HumanEval, synthetic tool-use) are fully public and cited with licence information. Tinker API credentials are *not* included for security reasons; the `README.md` explains how to obtain a Tinker API key.

6. **Experimental Setting/Details**

   (a) Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results? [Yes] Section 4 and Table 3 provide full hyperparameter settings for Modal experiments. Appendix B gives sweep ranges. Tinker API hyperparameters are provided in the API call scripts in the repository, but hardware details are outside our control and are noted as unknown. For Modal experiments, the GPU type (H100 SXM5, 80 GB), batch size, learning rate schedule, LoRA rank/alpha, and training duration are fully specified. Older Modal experiments used NVIDIA L4 GPUs (24 GB) as noted in Appendix A.

7. **Experiment Statistical Significance**

(a) Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments? [Partial] For Modal/TRL experiments with 5 seeds, we report mean ± SE with 95% bootstrap confidence intervals via `rliable` [5]. Statistical analysis is in progress; preliminary CIs are shown where seeds have completed. Tinker API experiments represent *single runs* (due to high failure rates) and are explicitly labelled as such; no statistical significance is claimed for Tinker-only comparisons. We follow the recommendations of **(author?)** [4] and **(author?)** [6] regarding the limits of single-seed evaluation.

8. **Experiments Compute Resources**

   (a) For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments? [Partial] Appendix A lists GPU types and estimated GPU-hours for each experiment group. The complete per-experiment breakdown, including wall-clock runtimes and estimated cost, is in `COMPUTE.md` in the repository.

   - **Modal experiments.** Hardware is fully specified: H100 SXM5 (80 GB) for recent runs, L4 (24 GB) for earlier experiments. Wall-clock runtimes and GPU-hour estimates are provided.
   - **Tinker API experiments.** The Tinker platform uses serverless GPU dispatch; exact GPU type is not disclosed by the API. Estimated GPU-hours are inferred from billing records and are labelled as approximate.

9. **Code Of Ethics**

   (a) Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics? [Yes] This work involves no human subjects, no private or sensitive data, and no models trained on proprietary corpora. All training datasets are publicly released research datasets (GSM8K, HumanEval, synthetic tool-use data). Broader societal impacts, including risks of reward hacking and alignment failure in RLHF systems, are discussed in the Broader Impact statement (Section 7.3 and `ethics_statement.tex`).

10. **Broader Impacts**

    (a) Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed? [Yes] Section 7.3 (and the accompanying `ethics_statement.tex`) discusses:

    - **Positive.** Lowering the barrier to RL post-training research; enabling reproducibility audits; surfacing platform-dependent variance that is often hidden in single-platform papers.
    - **Negative / risks.** RLHF reward hacking; alignment concerns when optimising proxy rewards; potential misuse of fine-tuned models; compute-access disparities that may limit replication to well-resourced groups.
    - **Carbon footprint.** Estimated 0.4–1.2 kg CO_2-eq for Modal H100 experiments; Tinker API footprint is unestimable due to undisclosed hardware.

11. **Safeguards**

    (a) Does the paper describe safeguards that have been put in place for responsible release of data or models that have been identified as requiring safeguards? [NA] The benchmark evaluates existing public models on mathematical reasoning, code generation, and synthetic tool-use tasks. No new capabilities with clear misuse potential are introduced; the fine-tuned models do not exhibit harmful behaviours beyond their base models. Released checkpoints are hosted on HuggingFace Hub under MIT licence with standard usage disclaimers.

12. **Licenses for existing assets**

    (a) Are the creators or original owners of assets (e.g., code, data, models) used in the paper properly credited, and are the licence and terms of use explicitly mentioned and properly respected? [Yes] All third-party assets are properly credited:

    - **GSM8K** [16]: MIT licence.
    - **HumanEval**: MIT licence (OpenAI).

- **Synthetic tool-use data**: self-generated from public API documentation; no licence restrictions.
- **Qwen model family**: Apache 2.0.
- **Llama model family**: Meta Community Licence.
- **TRL, PEFT, Transformers**: Apache 2.0 (HuggingFace).
- **Modal**: commercial platform; terms of service respected.
- **Tinker API**: proprietary; used under standard API terms of service.

13. **New Assets**

    (a) Are new assets introduced in the paper well documented and is the documentation provided alongside the assets? [Yes] New assets include: (i) the TINKERRL-BENCH benchmark suite (code, harness, evaluation scripts) released under MIT licence at `https://github.com/arvindcr4/tinker-rl-lab` with `README.md`, `REPRODUCE.md`, `ARTIFACT.md`, `COMPUTE.md`, `BASELINES.md`, and `BENCHMARKS_COMPARISON.md`; (ii) fine-tuned model checkpoints from successful Modal experiments hosted on HuggingFace Hub (`https://huggingface.co/arvindcr4/tinker-rl-bench-*`); (iii) all experiment logs on Weights & Biases (public project). Checkpoints from failed Tinker runs are *not* uploaded because those jobs did not produce valid final states.

14. **Crowdsourcing and Research with Human Subjects**

    (a) For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation? [NA] No crowdsourcing or human subjects research was conducted.

15. **Institutional Review Board (IRB) Approvals or Equivalent for Research with Human Subjects**

    (a) Does the paper describe potential risks incurred by study participants, whether compensation was provided to study participants, and whether informed consent was obtained? [NA] No human subjects research was conducted; IRB approval is not applicable.

16. **Declaration of LLM usage**

    (a) Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods? [Yes] LLMs (Qwen and Llama model families) are the primary *subjects* of evaluation; their use as RL fine-tuning targets is described in Sections 3 and 4. LLMs were **not** used to draft or revise this paper. Code-generation assistance from GitHub Copilot was used for boilerplate experiment scripts; no AI-generated content appears in the paper itself.